3Laws

Release beta

Thomas Gurriet (tgurriet@3laws.io)

Feb 09, 2024

1 Introduction **1**

2 Robot Diagnostic Module **3**
2.1 Installation . . . 3
2.2 Launch the RDM . . . 4
2.3 Configuration . . . 4

3 The Copilot **17**
3.1 Basic Architecture . . . 17
3.2 Available Configurations . . . 19
3.3 Applications . . . 20
3.4 Platforms . . . 21
3.5 Theory of Operation . . . 21
3.6 CoPilot Operational Modes . . . 22

Chapter 1

Introduction

Here is the public documentation for 3LawsRobotics Products:

- Robot Diagnostic Module
- The Copilot
- control_panel/index

Chapter 2

Robot Diagnostic Module

The **Robot Diagnostic Module (RDM) of 3LawsRobotics** is a powerful tool to gather analytics about your robot. It provides low level metrics such as CPU usage, signal delays but also high level safety critical metrics like safety envelope violation, planning inconsistency or sensor damage detection.

All of these metrics can be viewed on your web browser at 3laws.app.

In order to get your account setup and use freely this RDM, please contact support@3lawsrobotics.com

This product is still at a Beta development stage. New features are developed every month and some breaking changes can still happened. The 3LawsRobotics Team is available to answer your question.

For now only a ROS1, ROS2 interface is available but we intend to extend the offer of possible interfaces in the near future with support for MavLink or MQTT for instance.

2.1 Installation

The Robot Diagnostic Module is distributed as a debian package. A package is available for each of the following configuration

Ubuntu Distribution	ROS1 version	ROS2 version	Architecture
22.04	N/A	Humble/Iron	amd64/arm64v8
20.04	Noetic	Galactic/Foxy	amd64/arm64v8
18.04	Melodic	N/A	amd64/arm64v8

Table of Contents

2.1.1 . . . . . Quick Installation3
2.1.2. . . . . Non Interactive Installation3

2.1.1 Quick Installation

Here is the one liner that will download a public script that will, based on your system download the right package.

```
sudo bash -c "$(wget -qO -
https://raw.githubusercontent.com/3LawsRobotics/3laws/master/rdm/install.sh)"
```

2.1.2 Non Interactive Installation

If you want to install a specific package in a non interactive mode, here is the process:

- Download the script:

```
wget https://raw.githubusercontent.com/3LawsRobotics/3laws/master/rdm/install.sh
```

- Make it executable

```
chmod +x install.sh
```

- Run it with your arguments

```
sudo ./install.sh [-hyf] [-r <ROS_DISTRO>] [-a <ARCH>] [-v <UBUNTU_VERSION>]
```

if `-yf -r <ROS_DISTRO> -a <ARCH> -v <UBUNTU_VERSION>` specified, the script is non interactive

2.2 Launch the RDM

To launch the RDM, you only have to source your ros distribution:

```
source /opt/ros/<DISTRO>/setup.sh
```

and to run the command:

```
ros2 launch lll_rdm rdm.launch.py
```

You can add the following command line options:

- **use_sim_time**: <Boolean, default: false> Specify if the RDM use the ros time or the machine time
- **config_filename**: <String> Path to the config file if not in the */opt/3lawsRoboticsInc/robot_diagnostic_module/config* folder (**needs to start with /**) or just the config file name inside the default folder.
- **robot_id**: <String> name of your robot
- **log_level**: <trace | debug | info | error, default: info> Log level of the RDM
- **dry_run**: <Boolean> Start the RDM, check the config file and turn off without error if everything has been launched correctly

2.3 Configuration

Table of Contents

2.3.1 . . . . . Configuration Template5
2.3.2 . . . . . General configuration rules5
2.3.3 . . . . . Scheme6

The configuration is written inside a YAML. file. This file can be stored anywhere but the default folder is `/opt/3lawsRoboticsInc/robot_diagnostic_module/config`

Other path can be specified in the command line launch command

The Robot Diagnostic Module is shipped with a configuration template that can be found here

`/opt/3lawsRoboticsInc/robot_diagnostic_module/config/template_config.yaml`

In order to have the more user friendly configuration experience, a lot of the fields are optional. The configuration can be build alongside your robot so that it does not represent a burden for the developer.

2.3.1 Configuration Template

You can find here a complete template configuration.

- examples/template_example

2.3.2 General configuration rules

Bounds

All bounds parameters are similar:

- **norm_type**: Norm to use for norm bound, possible values (none, L_1, L_2, L_infinity)
- **norm_upper_bound**: Upper norm bound, floating point
- **norm_lower_bound**: Lower norm bound, floating point
- **upper_bounds**: Upper bounds on the value of the components of the tracking_error, sequence of floating point
- **lower_bounds**: Lower bounds on the value of the components of the tracking_error, sequence of floating point
- **rates_upper_bounds**: Upper bounds on the value of the components of the tracking_error, sequence of floating point
- **rates_lower_bounds**: Lower bounds on the value of the components of the tracking_error, sequence of floating point

Set both (rate_)upper_bounds[i] and (rates_)lower_bounds[i] to .inf to disable bound on component i.

Leave empty to deactivate all (rate) bounds checking.

Both (rate_)upper_bounds and (rates_)lower_bounds must have same size

Duration parameters

Parameters like signal_min_rate represent durations. They can be specified by an integer number corresponding to a number of nanoseconds:

- **signal_min_rate**: 1 -> 1 nanoseconds
- **signal_min_rate**: 1000000000 -> 1 second

They can also be specified by an integer of floating point followed by the time scale:

- **signal_min_rate**: 1ns -> 1 nanoseconds
- **signal_min_rate**: 25us -> 25 microseconds
- **signal_min_rate**: 0.5ms -> 500 milliseconds
- **signal_min_rate**: 1s -> 1 second
- **signal_min_rate**: 20hz -> 50 milliseconds
- **signal_min_rate**: 1min -> 1 minute
- **signal_min_rate**: 1h -> 1 hour
- **signal_min_rate**: 1d -> 1 day
- **signal_min_rate**: 1y -> 1 year

2.3.3 Scheme

Credentials

Credentials are used to connect to the databases to store the data. Here is the structure and the field detail.

```
credentials:
  company_id: Company name
  robot_id: Robot Name
  influx_credentials:
    database_token: INFLUX token provided by 3lawsRobotics
  clickhouse_credentials:
    username: Company name
    password: CLICKHOUSE password provided by 3lawsRobotics
```

Diagnostic Configuration

This part of the configuration is here to define diagnostic specific options.

Legend:

- <Distance array> An array of distance in meter
- <Percentage array> An array of percentages (between 0 and 100)
- <Time array> An array of time. Allowed unit: s, ms, min
- <Ratio Array> An array of ratio (between 0 and 1). Used when values are normalized
- <Severity> An enum: ok | minor | severe | critical | ~

If a value is set to [] or ~ the RDM will not flag event for this specific case. In the case of an array option, the first value is the threshold to detect a critical event, the second for a severe event and so on. So for example, a Distance Array like this [0.001, 0.005, 0.01] will flag a minor event if the value goes under 0.01, then reevaluates it to 0.005 if the value is inferior to 0.005 and finally raise the severity to critical if the value goes under 0.001

```
diagnostic_config:
  timeout_factor: <Int> Specify how much time the max signal rate has to be wait
                        to declare a topic timed out. This trigger an event.
  max_signals_delay: <Time <int> ms/s/min> Specify the maximum delay between
                                           the send time and the received time
  incident_detection: This structure specify the severity and the threshold for
the event metrics
    min_event_time: <Time <int> ms/s/min> Minimal time to flag an event
    min_displayed_severity: <Enum: ok|minor|severe|critical> Minimal criticality
to flag an event
    collision:
      limit_dists: <Distance array>
    computer:
      limit_cpu_loads: <Percentage array>
      limit_disk_usages: <Percentage array>
      limit_ram_usages: <Percentage array>
    clock:
      limit_utc_deviations: <Time array>
      limit_rtc_deviations: <Time array>
    node_health:
      timeout_severity: <Severity>
      not_ok_severity: <Severity>
      limit_delay_bounds: <Ratio Array>
      limit_rate_bounds: <Ratio Array>
    obstruction:
      obstruction_severity: <Severity>
```

```
    signal_health:
      nan_severity: <Severity>
      zero_severity: <Severity>
      subnormal_severity: <Severity>
      inf_severity: <Severity>
      bad_timestamp_severity: <Severity>
      limit_bounds: <Ratio Array>
    dynamic:
      limit_xdot_difference: <Ratio Array>
      input_domain_severity: <Severity>
      state_domain_severity: <Severity>
    tracking:
      limit_tracking_difference: <Ratio Array>
  upload: This configuration allow the user to reduce the bandwidth
          by not uploading high frequency metrics
    high_frequency:
      state: <Boolean>
      input: <Boolean>
      path_tracking_error: <Boolean>
      control_tracking_error: <Boolean>
```

Interface Configuration

The interface structure is here to get precision on the interface. Right know only ROS interfaces are available, the interface option are here to specify Quality of Service and topic types

```
interface:
  retimestamp: never/always/if_zero specify if the timestamp has to be set

  ros_topics_info:
    /example_topic!:
      type: sensor_msgs/LaserScan
    /example_topic2:
      type: lll_msgs/Float64VectorStamped
    /example_topic3:
      type: nav_msgs/Odometry
```

Supported topics type are all ROS default types and 3lawsRobotics custom types (ns: lll_msg, lll_rdm_msg)

Robot Description

This part is the most important one. The more accurate the information provided here, the more insightful the RDM will be.

Each sub structure is optional but once a substructure is declared, all its fields must be filled (unless Optional is specified)

Table of Contents

Robot model 7
Auto Generated Nodes 10
Robot Autonomy stack 10
Robot Perception 11
Robot control 13
Extras 15

Robot model

```
robot_type: <String> See available robot type section for more details

kinematic_model: # Kinematic model of the robot
  base_frame_id: <String> # Name of the base frame of the robot
  geometry: # Geometry of the robot
    shape_type: <Enum point|box|sphere|ellipsoid|capsule|cone|cylinder|mesh> #
Type of the geometry shape (should be one of [point, box, sphere, ellipsoid,
```

```
capsule, cone, cylinder, mesh])
    shape_params: # Parameters for the chosen shape_type
      x_size: <Float> #in Meter
      y_size: <Float> #in Meter
      z_size: <Float> #in Meter
    pose: # [optional] Pose of shape w.r.t base frame
      translation: <Float array[3]> # [optional] Translation w.r.t base frame
[x,y,z], default [0,0,0]
      rotation: <Float array[4]> # [optional] Rotation quaternion w.r.t base frame
[w,x,y,z], default [1,0,0,0]

dynamical_model: # [optional] Behavior model of the robot
  model_type: <String> # Type of dynamical model for selected robot_type
  state_domain: # Region of the state space in which the dynamical model has been
validated
    simple: <Boolean> # [optional], If true, assumes M is an identity matrix
(default: false)
    n_cstr: 3 # [optional if simple==true] Number of rows in M
    M: # [optional if simple==true] Either a single sequence representing
the diagonal of M, or a sequence of the rows of M
      - [1., 0., 0., 0., 0.]
      - [0., 1., 0., 0., 0.]
      - [0., 0., 1., 0., 0.]
    # Defined as a a convex polytope S={x in Rn | lb <= M.x <= ub}
    ub:
      <Float array[n_cstr]> # If simple==true, must have same size as state of
model_type,
      # otherwise must have size n_cstr
    lb:
      <Float array[n_cstr]> # If simple==true, must have same size as state of
model_type,
      # otherwise must have size n_cstr

    input_domain: # Region of the input space accessible for control purposes and
in which the dynamical model has been validated
      # Same representation as 'state_domain'
      ub:
        <Float array[model_input_size]> # If simple==true, must have same size as
input of model_type,
        # otherwise must have size n_cstr
      lb:
        <Float array[model_input_size]> # If simple==true, must have same size as
input of model_type,
        # otherwise must have size n_cstr
      simple: <Boolean>

  process_noise_covariance:
    <Float array[model_state_size]> # Either a single sequence representing
the diagonal of the process noise covariance matrix,
    # or a sequence of its rows. Must have same size as state of model_type
  model_param: # [variant] Parameters for the chosen model_type, as example here
for the bicycle
    wheel_dx: 1.
    origin_dx: 1.
    tau_vel: 1.
    tau_steer: 1.
```

Available shape types

The kinematic model of the robot defines its geometry. The geometry is described by a base frame, a shape, and the pose of that shape in the base frame. We currently support the following shapes:

- **Point**:

```
shape_params: ~
```

- **Box**:

```
shape_params:
  x_size: <Float>  Total length of the box along x axis
  y_size: <Float>  Total length of the box along y axis
  z_size: <Float>  Total length of the box along z axis_mask
```

- **Sphere**:

```
shape_params:
  radius: <Float>  Radius of the sphere
```

- **Ellipsoid**:

```
shape_params:
  radius_x: <Float>  Semi x-axis length
  radius_y: <Float>  Semi y-axis length
  radius_z: <Float>  Semi z-axis length
```

- **Capsule**:

```
shape_params:
  radius: <Float>  Radius of the capsule
  length: <Float>  Length of the capsule
```

- **Cone**:

```
shape_params:
  radius: <Float>  Radius of the cone
  length: <Float>  Length of the cone
```

- **Cylinder**:

```
shape_params:
  radius: <Float>  Radius of the cylinder
  length: <Float>  Length of the cylinder
```

- **Mesh**:

```
shape_params:
  mesh_file: /opt/mesh.stl  Path to mesh file
  mesh_type: stl  Type of mesh file, available options: [stl]
  mesh_units: mm  [optional] Units of the mesh file, available options: [mm, cm,
dm, m, dam, hm, km, mi, nm, yd, ft, in], default: m
```

Available robot type

Each dynamical model type has its own set of states, inputs, and parameters:

- mobile_robot:
  - differential_drive: Dynamical model for a rigid body over SE2 with first order tracking response of longitudinal and rotational body velocities
    states: [x,y,yaw,vx_body_actual,wz_body_actual]
    inputs: [vx_body_command,wz_body_command]
    parameters:

    - tau_vel: time constant of the 1st order tracking response in linear velocity (1/s) (must be strictly positive)
    - tau_yaw_vel: time constant of the 1st order tracking response in angular velocity (1/s) (must be strictly positive)

- bicycle: Dynamical model for a 2 wheels or 4 wheel but with coupled front wheel steering vehicle over SE2, with first order tracking response of steering angle and origin velocity magnitude.
  states: [x,y,yaw,||v_body||_actual,steering_angle_actual]
  inputs: [||v_body||_command,steering_angle_command]
  parameters:
    - wheel_dx: Distance between front and back wheels (m) (must be strictly positive)
    - origin_dx: Position of vehicle's origin w.r.t back wheels (m) (must be positive)
    - tau_vel: time constant of the 1st order tracking response in linear velocity (1/s) (must be strictly positive)
    - tau_steer: time constant of the 1st order tracking response in angular velocity (1/s) (must be strictly positive)

Auto Generated Nodes

The RDM will automatically create node based on the configuration. So for each substructure of robot_description, a node will be created. For example a node sensor is created and will group all topics from each sensor declared in the configuration.

If you want to add extra topic to a node, you can declare them via the **extra_topics** structure. This structure is a list of topics.

A topic has to be declared like this:

```
- interface_id:
    /test_node_topic_1 # Name of the ros topic. Associated
'interface.message_type_map.<interface_id>' must be specified.
    # Supported types: [builtin_interfaces/*, geometry_msgs/*, lll_msgs/*,
nav_msgs/*, rcl_interfaces/*, rosgraph_msgs/*, sensor_msgs/*, std_msgs/*,
trajectory_msgs/*, visualization_msgs/*]
  topic_id: test_node_topic_1 # Display name for this topic, must be UNIQUE among
all topics of each node
  signal_min_rate: 1s # Maximum allowed duration without receiving data
```

If a topic is declared like this, inside the interface section, **the message_type has to be declared**.

Another information can be added to a node: its process_name. When a process name is specified, the RDM will find the process and monitor his CPU and network usage.

Robot Autonomy stack

Mission Manager

```
mission_manager:
  extra_topics: ~ # [optional]
  process_name: ~ # [optional]
  finite_states: # Finite states of the robot
    - interface_id:
        /status # Name of the ros topic.
        # Supported types: [(default) std_msgs/String, std_msgs/FloatXX,
std_msgs/Bool, std_msgs/Char, std_msgs/Byte, std_msgs/IntXX, std_msgs/UIntXX]
      sender_id: state_machine # Display name for sender of this state
      state_id: status # Identifier for this state, "<sender_id>.<state_id>" must
form a UNIQUE identifier among all signals
      signal_min_rate: 1s
    - interface_id: /search_mode
```

```
      sender_id: state_machine
      state_id: search_mode
      signal_min_rate: 1s
```

Path Planning

```
path_planning:
  extra_topics: ~ # [optional]
  process_name: ~ # [optional]
  paths:
    - interface_id:
        /desired_path # Name of the ros topic.
        # Supported types: [(default) lll_msgs/Trajectory, nav_msgs/Path,
trajectory_msgs/JointTrajectory]
      path_id: main_path # Display name for this path, must be UNIQUE among all
paths
      trajectory_state_size: 7 # Size of the trajectory state vector
      signal_min_rate: 1min # Maximum time without receiving data before signal is
considered timed out
      # state_mask:
      #   [0, 1, 2, 3, 4, 5, 6] # [optional] If the path only corresponds to
a subset of the state_estimation vector,
      #   # use this mask to extract the relevant data : trajectory_state[i] =
state_estimation[state_mask[i]].
      #   # Must be of size 'trajectory_state_size', and not contain indices
greater than state_estimation.state_size.
      #   # If not specified or null, will be [0, ..., trajectory_state_size-1]
      tracking_error_bounds: ~ # [optional] Bounds on controller's tracking error
: path_state - actual_state
```

Robot Perception

Sensors:

```
sensors:
    extra_topics: ~ # [optional]
    process_name: ~ # [optional]
    batteries: [] # Coming soon!
    cameras: [] # Coming soon!
    gps: [] # Coming soon!
    imus: [] # Coming soon!
    lidars: [] # Coming soon!
    laserscans: # See next section
    loadcells: # See next section
```

Laserscan

```
laserscans: # Planar laser scanners
  - interface_id:
      /laserscan_1_topic # Name of the ros topic.
      # Supported types: [(default) sensor_msgs/LaserScan]
    sensor_id: laserscan_1 # Display name for this laserscan, must be UNIQUE among
all laserscans
    signal_min_rate: 1s # Maximum time without receiving data before signal is
considered timed out
    specs:
      n_rays: 1000 # Expected number of rays in the laserscan
      angle_min: -3.14 # Minimum ray angle
      angle_max: 3.14 # Maximum ray angle
      range_min: 0. # Minimum ray range
      range_max: 1000. # Maximum ray range
      noise_one_sigma: 0.025 # Expected standard_error of the sensor (given by
the manufacturer, often like: precision = +-2sigma)
    transform: # Specification of frame w.r.t which the measurement is expressed
      parent_frame_id: robot # Id of parent frame
```

```
      pose: # [optional] Pose w.r.t parent frame
        translation: [0., 0., 0.] # [optional] Translation w.r.t parent frame
[x,y,z], default [0,0,0]
        rotation: [1., 0., 0., 0.] # [optional] Rotation quaternion w.r.t parent
frame [w,x,y,z], default [1,0,0,0]
```

Loadcells

```
loadcells: # Force and torque measurement sensor, 6 axis by default
  - interface_id:
      /end_effector_wrench # Name of the ros topic.
      # Supported types: [(default) lll_msgs/Float64VectorStamped, any other
vectorizable type (see bottom of this file)]
    sensor_id: end_effector_loadcell # Display name for this loadcell, must be
UNIQUE among all loadcells
    signal_min_rate: 1s # Maximum time without receiving data before signal is
considered timed out
    transform: # Specification of frame w.r.t which the measurement is expressed
      parent_frame_id: robot # Id of parent frame
      pose: # [optional] Pose w.r.t parent frame
        translation: [0., 0., 0.] # [optional] Translation w.r.t parent frame
[x,y,z], default [0,0,0]
        rotation: [1., 0., 0., 0.] # [optional] Rotation quaternion w.r.t parent
frame [w,x,y,z], default [1,0,0,0]
    # axis_mask: # [optional] Define which of the 6 force/torque axes in SE3
the loadcell signals correspond to: [Fx, Fy, Fx, Mx, My, Mz].
    #   # If not specified or null, assumes all 6 axes.
    #   # Cannot be empty or longer than 6. Index must be between 0 and 5
included.
    #   [0, 5] # Corresponds to a 2 axis loadcell [Fx,Mz]
    noise_one_sigma: [1., 1., 1., 1., 1., 1.] # Noise characteristics of loadcell
axes. Must have same size as axis_mask
    bounds: ~ # [optional]
```

Perception

```
perception:
  obstacles: # [optional] List of obstacles
    interface_id: /obstacles # Name of the ros topic. # Supported types:
[(default) lll_msgs/ObjectArray]
    signal_min_rate: 1s # Maximum time without receiving data before signal is
considered timed out
    meshes: # List of meshes to be loaded
      []
      # - id: sphere # Mesh identifier, must be UNIQUE among all meshes
      #   data: # Mesh data
      #     mesh_file: sphere.stl # Path to mesh file
      #     mesh_type: stl # Type of mesh file
      #     mesh_units: mm # Unit of mesh file
```

Localization

```
localization:
  extra_topics: ~ # [optional]
  process_name: ~ # [optional]
  state_estimation: # [optional] See next section
  odometry: # [optional] See next section
```

State Estimation

```
state_estimation:
  interface_id:
    /state # Name of the ros topic.
    # Supported types: [(default) lll_msgs/Float64VectorStamped, any other
vectorizable type (see bottom of this file)]
```

```
  signal_min_rate: 1s # Maximum time without receiving data before signal is
considered timed out
  state_size: 5 # Size of the state vector
  # mask:
  #   [0, 1, 2, 3, 5] # [optional] If only a subset of the vectorized message
actually constitute the state vector
  #   # use this mask to extract the relevant data : state[i] =
msg_vectorized[mask[i]].
  #   # Must be of size 'state_size', and not contain indices greater than
the size of vectorized message.
  #   # If not specified or null, will be [0, ..., state_size-1]
  bounds: ~ # [optional]
```

Odometry

```
odometry:
  - interface_id:
      /odom_node_0_topic # Name of the ros topic.
      # Supported types: [(default) nav_msgs/Odometry]
    odom_id: odom_node_0 # Display name for this odometry source, must be UNIQUE
among all odometry
    signal_min_rate: 1s # Maximum time without receiving data before signal is
considered timed out
    se2_only: true # [optional] Consider only SE2 projection of pose and twist
(default: false)
    position_bounds: # [optional] Position part of the odometry. Components are
[x,y,z] or [x,y] if se2_only==true
      norm_type: none
      norm_upper_bound: 1.
      norm_lower_bound: 0.
      upper_bounds: [1., 1.]
      lower_bounds: [-1., -1.]
      rates_upper_bounds: []
      rates_lower_bounds: []
    orientation_bounds: ~ # [optional] Same fields as position. Components are
[roll,pitch,yaw] or [yaw] if se2_only==true
    velocity_linear_bounds: ~ # [optional] Same fields as position. Components are
[vx,vy,vz] or [vz,vy] if se2_only==true
    velocity_angular_bounds: ~ # [optional] Same fields as position. Components
are [wx,wy,wz] or [wz] if se2_only==true
```

Robot control

```
control:
  extra_topics: ~ # [optional]
  process_name: ~ # [optional]
  setpoint_tacking_controllers: # See next section
  actuators: # See next section
  supervisors: # See next section
```

SetPoint Tacking Controller

```
setpoint_tacking_controllers: # PID like controllers
  - controller_id: velocity_controller # Display name for this controller, must be
UNIQUE among all controllers
    state_size: 1 # Size of controller setpoint
    input_size: 1 # Size of control input computed by controller
    desired_state:
      interface_id: /controller_cmd_topic # Name of the desired state ros topic.
      # Supported types: [(default) lll_msgs/Float64VectorStamped, any other
vectorizable type (see bottom of this file)]
      mask: [0] # [optional] If only a subset of desired_state_topic_id vector is
actually used by controller,
      # use this mask to extract the relevant data : desired_state_used[i] =
```

```
desired_state_received[desired_state_mask[i]]
      signal_min_rate: 1s # Maximum time without receiving data before signal is
considered timed out
      bounds: ~ # [optional] Bounds on desired state

    actual_state:
      interface_id: /controller_state_topic # Name of the actual state ros topic.
      # Supported types: [(default) lll_msgs/Float64VectorStamped, any other
vectorizable type (see bottom of this file)]
      mask: ~ # [optional] Same as desired_state_mask
      signal_min_rate: 1s # Maximum time without receiving data before signal is
considered timed out
      bounds: ~ # [optional] Bounds on actual state

    control_input:
      interface_id: /controller_input_topic # Name of the control input ros topic.
      # Supported types: [(default) lll_msgs/Float64VectorStamped, any other
vectorizable type (see bottom of this file)]
      mask: ~ # [optional] Same as desired_state_mask
      signal_min_rate: 1s # Maximum time without receiving data before signal is
considered timed out
      bounds: # [optional] Bounds on controller's control input
        norm_type: none
        norm_upper_bound: 1.
        norm_lower_bound: 0.
        upper_bounds: []
        lower_bounds: []
        rates_upper_bounds: [1.]
        rates_lower_bounds: [-1.]

    tracking_error_bounds: # [optional] Bounds on controller's tracking error :
desired_state - actual_state
      norm_type: none
      norm_upper_bound: 1.
      norm_lower_bound: 0.
      upper_bounds: [1.]
      lower_bounds: [-1.]
      rates_upper_bounds: []
      rates_lower_bounds: []
```

Actuators

```
actuators: # Robot actuation
  combined: # Combined actuation vector
    interface_id:
      /control_input # Name of the ros topic publishing the complete robot
actuation vector.
      # Supported types: [(default) lll_msgs/Float64VectorStamped, any other
vectorizable type (see bottom of this file)]
    input_size: 3 # Size of the combined input vector
    signal_min_rate: 1s # Maximum time without receiving data before signal is
considered timed out
    bounds: ~ # [optional]
    # mask:
    #   [0,1,3] # [optional] If only a subset of the vectorized message actually
constitute the combined input vector
    #   # use this mask to extract the relevant data : input[i] =
msg_vectorized[mask[i]].
    #   # Must be of size 'input_size', and not contain indices greater than
the size of the vectorized message.
    #   # If not specified or null, will be [0, ..., input_size-1]
```

Supervisors

```
supervisors: # 3Laws AI Supervisors
  - interface_id: /main_supervisor_topic # Name of the supervisor data ros topic.
    supervisor_id: main_supervisor # Display name for this supervisor, must be
UNIQUE among all supervisor
    signal_min_rate: 1s # Maximum time without receiving data before signal is
considered timed out
```

Extras

```
extras:
  passthrough_metrics: # See next section
  clocks: # See next section
  signals: # See next section
  nodes: # See next section
```

Passthrough Metrics

```
passthrough_metrics: # Generic passthrough for scalar metric signals
  - interface_id:
      /metric_1_topic # Name of the ros topic.
      # Supported types: [(default) std_msgs/Float64, std_msgs/Float32,
std_msgs/Bool, std_msgs/Char, std_msgs/Byte, std_msgs/IntXX, std_msgs/UIntXX]
    metric_id: metric_1 # Display name for this metric, must be UNIQUE among all
passthrough metrics
    metric_group_id:
      position # [optional] Group this signal belongs to.
      # Metrics of the same group are plotted on the same graph in 3laws.app
```

Clocks

```
clocks:
  - interface_id:
      /custom_clock # Name of the ros topic.
      # Supported types: [(default) rosgraph_msgs/Clock]
    clock_id: my_clock # Display name for this clock, must be UNIQUE among all
clocks
    signal_min_rate: 1s # Maximum time without receiving data before signal is
considered timed out
```

Signals

```
signals: # Generic floating point multidimensional signal values sanity and bounds
checking
  - interface_id:
      /test_signal_topic # Name of the ros topic.
      # Supported types: [(default) lll_msgs/Float64VectorStamped, any other
vectorizable type (see bottom of this file)]
    sender_id: test_signal_node # Display name of sender node
    signal_id: test_signal # Display name of this signal,
"<sender_id>.<signal_id>" must form a UNIQUE identifier among all signals
    signal_size: 1 # Size of this signal
    signal_min_rate: 1s # Maximum time without receiving data before signal is
considered timed out
    # mask:
    #   [2] # [optional] If only a subset of vectorized message actually
constitute the signal vector
    #   # use this mask to extract the relevant data : signal[i] =
msg_vectorized[mask[i]].
    #   # Must be of size 'signal_size', and not contain indices greater than
the size of the vectorized message.
    #   # If not specified or null, will be [0, ..., signal_size-1]
    bounds: ~ # [optional]
```

Nodes

```
nodes: # Generic node health checking metric
  - node_id: test_node # Display name of node, must be UNIQUE among all nodes
    # text_log_interface_id:
    #   /test_node_log # [optional] Name of the ros topic publishing log info for
that node.
    #   # Supported types: [(default) rcl_interfaces/Log]
    # process_name: # [optional]
    #   test_node_exec.
    topics: # List of topics published by the node (only available in ros2 humble
and up)
      - interface_id:
          /test_node_topic_1 # Name of the ros topic. Associated
'interface.message_type_map.<interface_id>' must be specified.
          # Supported types: [builtin_interfaces/*, geometry_msgs/*, lll_msgs/*,
nav_msgs/*, rcl_interfaces/*, rosgraph_msgs/*, sensor_msgs/*, std_msgs/*,
trajectory_msgs/*, visualization_msgs/*]
        topic_id: test_node_topic_1 # Display name for this topic, must be UNIQUE
among all topics of each node
        signal_min_rate: 1s # Maximum allowed duration without receiving data
  - node_id: rosout
    text_log_interface_id: /rosout # If equal to '/rosout', uses 'name' field of
incoming rcl_interfaces/Log message as node_id for text_log message
    topics: []
```

The configuration has **4** distinct parts

- Credentials:
  This part is here to specify the credentials to access the databases. These credentials are provided to you by email once your 3LawsRobotics account is created.
- Diagnostic Configuration:
  This part is the more important one. It describe your robot desired behavior, the interface data stream access for the RDM to monitor.
- Interface Configuration:
  Here are specified the diagnostic specific options such has the criticality of a certain event or the threshold above which a incident is flagged.
- Robot Description:
  And this last part allow the user to specify interface specific options such has ROS Quality of Service.

Chapter 3

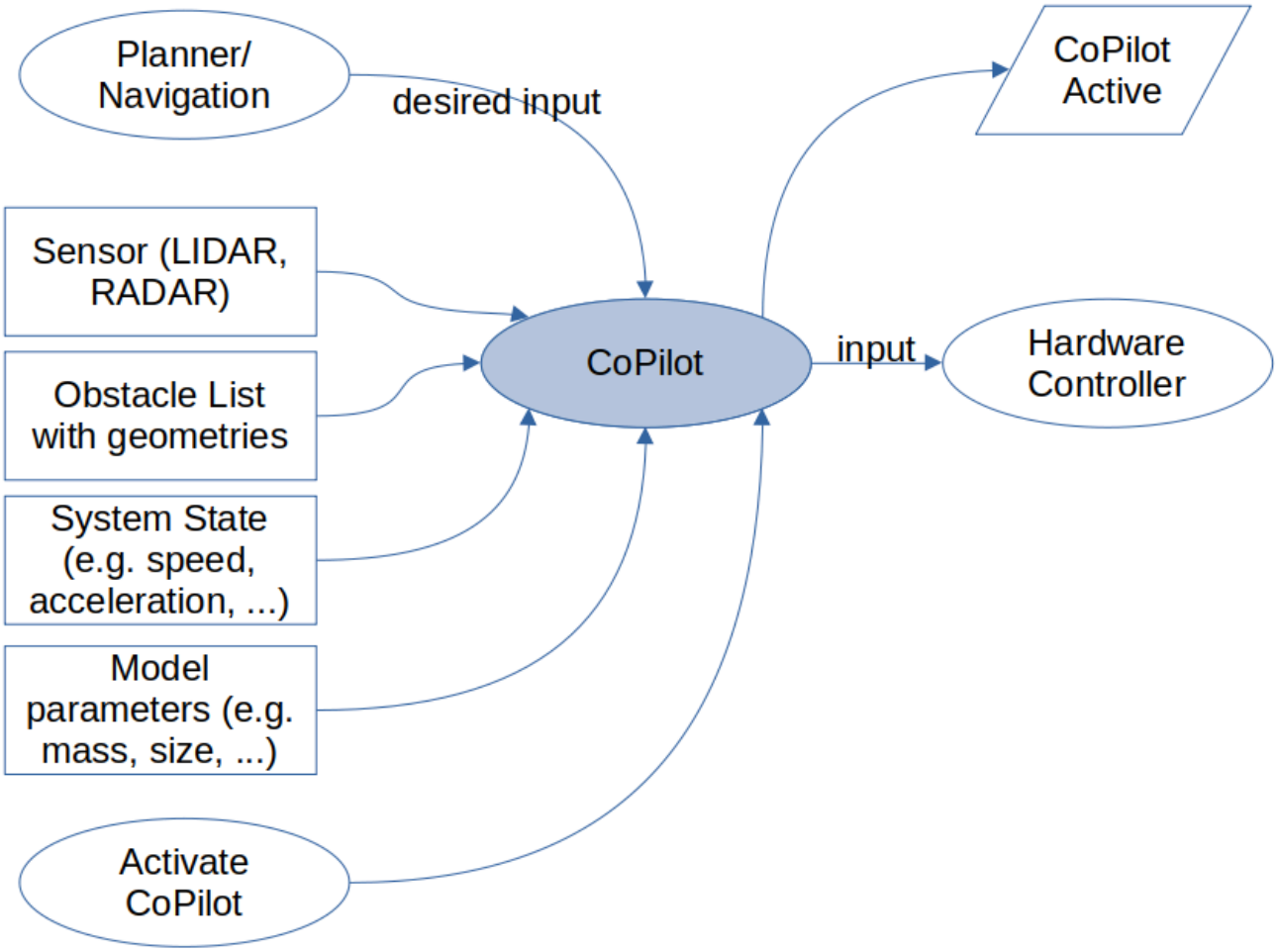
The Copilot

3Laws provides the *Copilot* as an add-on that can check and augment an existing planner or controller. The high-level idea is that the Copilot monitors the robot's state and next motion commands in real-time. If the model predicts that the system will experience undesirable behavior (based on the commands and system dynamics), then the commands will be altered to avoid the undesirable outcome in a minimally disruptive way.

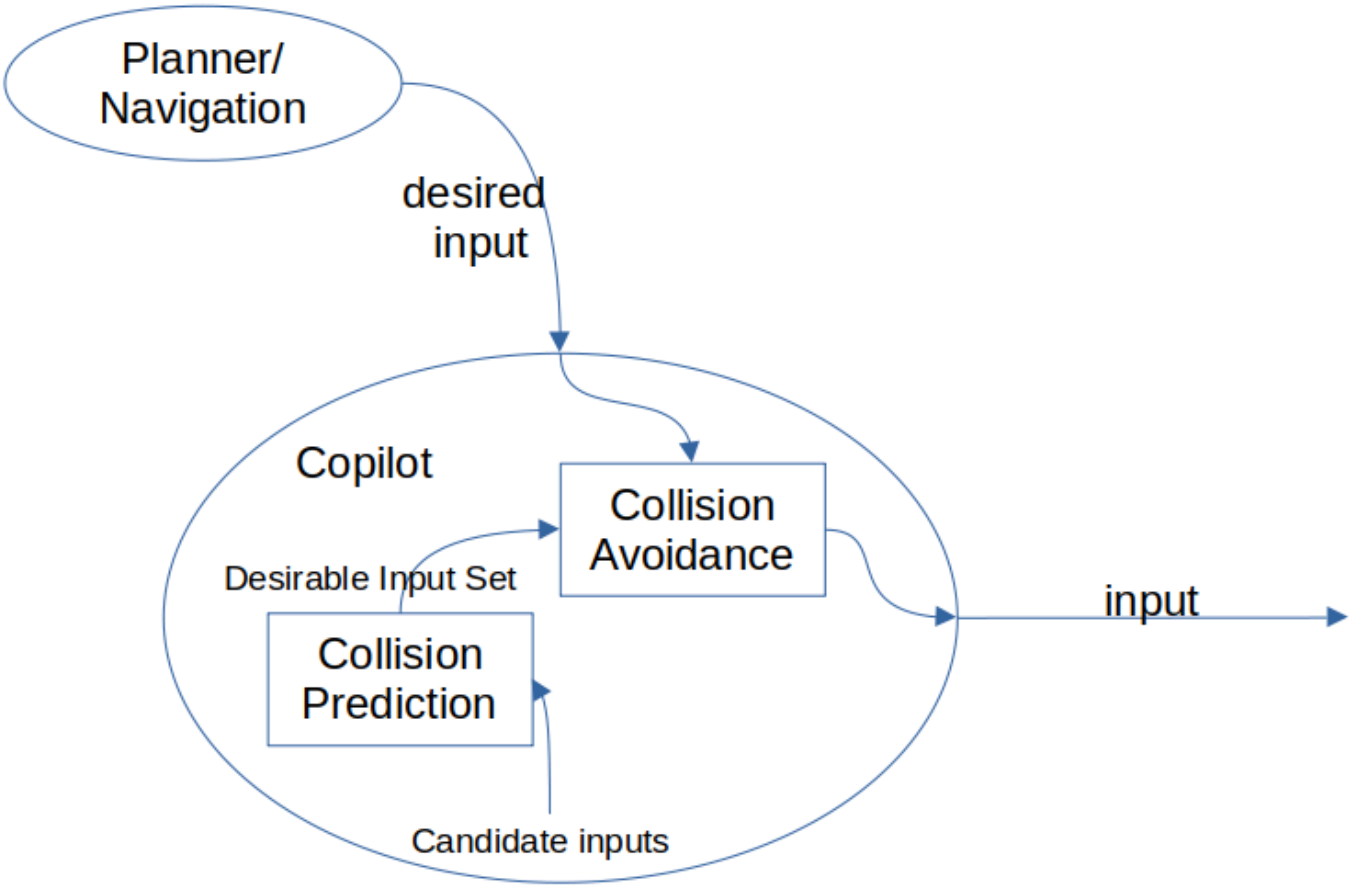
The off-the-shelf packaging of *Copilot* supports a few pre-built operational situations such as collision avoidance for a selection of robot systems such as unicycle (differnential- drive) and bicycle (front-steering) vehicles. Other operational situations like geofencing or attitude stabilization can be implemented as custom behaviors. Similarly, other operational platforms including articulated robots, aerial and marine vehicles, and specialized devices can also be implemented by 3Laws.

3.1 Basic Architecture

From an operational standpoint, the CoPilot (when used for collision avoidance) sits between the planning layer and the hardware control layer. "Hardware control" typically refers to a speed or attitude controller for a vehicle or a joint attitude/speed controller for an articulated robot.



The CoPilot will evaluate the likelihood of a collision (or other undesirable behavior) and will only modify the desired input from the planner when a collision is predicted within the estimation window.



There are 2 main steps to integrate CoPilot into an existing stack:

1. Remap the output from the planner (or component that produces commands such as the path to follow, the vehicle speed, or the navigation) and set it as the input to the CoPilot. Then remap the input of the controller (or component that converts the navigation instructions into hardware/ actuator instruction to use the output of CoPilot. With ROS, for example, the re-mapping can be done in the launch routines.
2. Start the CoPilot as part of the stack.

3.2 Available Configurations

The following configurations of robot platform and operational objective are currently available.

Robot Configuration	Collision Avoidance	GeoFencing
Unicycle	in development	
Omnidirectional	in development	
Bicycle	in development	
Copter Drone	in development	in development

In terms of handling the objects in the space, these routines support laser scans and list of obstacles (with geometries). Using lists of perceived obstacles typically results in lower computational loads.

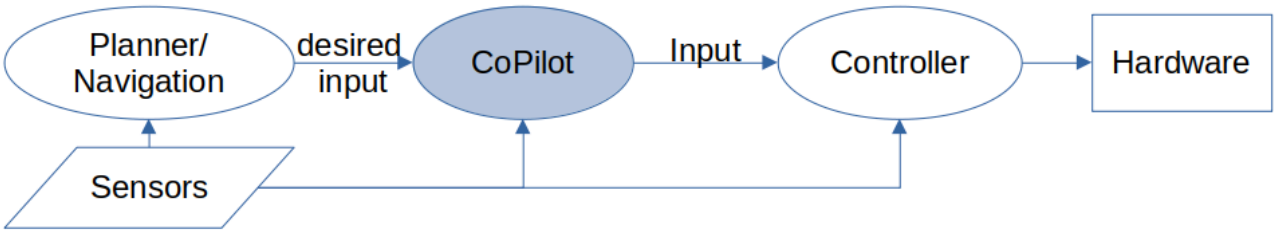
3.3 Applications

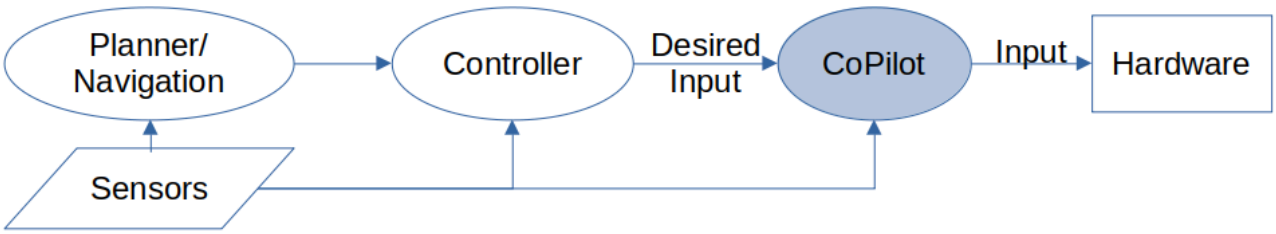
Active Collision Avoidance: In the collision avoidance use case, the CoPilot is designed to modify the planner's outputs in order to prevent impacts between the robot and other actors/obstacles in the operational space. CoPilot can be used as a redundant system that filters the planner and only injects changes when a collision is predicted. In many cases, this allows for reducing the load on the planner for calculating paths around obstacles or for using the CoPilot as a redundant component in the stack, allowing improved reliability calculations by having a dissimilar component that can perform obstacle handling as a parallel task. A sensor system that detects the presence of the obstacles (e.g. LIDAR, RADAR, ultra-sonic) is required for collision avoidance. The sensor suite can be shared with the stack or can be dedicated.

In the case of a human "planner", the CoPilot can be responsible for avoiding obstacles (assuming they are detected by sensors). Avoidance actions that can be activated in the CoPilot include stopping, avoiding the obstacle to the left/right, and backing up. Custom actions (such as diverting and parking until a moving obstacle on a known trajecotry has passed by) can also be implemented.

Repetitive Path Collision Avoidance: Using a simulation framework with known obstacles in the space, Copilot can modify pre-planned paths to avoid potential collisions. The margins between the robot (even for articulated robots) and the obstacles can be adjusted based on estimated uncertainties in the behavior of the robot and of positioning of the fixed objects. The primary use case for this is for systems that perform repetitive tasks.

GeoFencing: In previous deployments, the CoPilot has been inserted into the autonomy stack at different locations based on the needs for time-criticality. CoPilot it most often deployed between the planner and the inner-loop controller, but it can also be deployed between the inner-loop controller and the hardware for cases where the vehicle, like a racing copter, is going to approach undesirable configurations at rates that the high-level planner is able to replan-for, or if the system is being controlled by a human (or automated planner) that is not aware of the position of the drone relative to the keep-out area.





Configuration Bounding: CoPilot's underlying math is designed to control state variables such as position, speed, and accelerations. This means that configuring CoPilot to avoid situations such as vehicle roll-over because of large lateral accelerations or sliding because of large accelerations can also be implemented as objectives. Please contact 3Laws for discussions on how these objectives can be

made available.

3.4 Platforms

3Laws is pre-packaging several combinations of platform (e.g. robot mechanical layout) and Application. These are the most common use-cases that 3Laws is aware of.

Unicycle describes a wheeled-ground-based robot with differential drive for steering and coordinated drive for forward/back motion. The vehicle is able to stop and rotate in-place. Configuration parameters include wheel radius, distance between the wheels, vehicle extents, acceleration limits, and speed limits.

Bicycle includes vehicles that can be modeled with a single-track rolling model (e.g. car, truck, golf-cart). Current models use front-wheel steering. Control consists of speed and steering. Configuration parameters include wheel radius, maximum steering angles, effective wheelbase, vehicle extents, vehicle mass, acceleration limits, speed limits, and for faster vehicles, understeer gradient.

Omnidirectional robots can move longitudinally and laterally, often at the same time.

Copter: Flying vehicle that can move and rotate freely in a 3-dimensional world, but must be upright most of the time to avoid colliding with the ground. Configuration parameters include vehicle extents, mass, moments of inertia, acceleration limits, and speed limits.

3.5 Theory of Operation

The 3Laws Copilot is a product that uses theories from *invariant set* math for the states of systems to create a mechanism to keep the devices away from undesiread state configurations (e.g. unsafe areas, unstable configurations). For systems that are controlled through feedback or feedforward, the desirable state is based on the needs of the operation and what sensing/actuation methods are available. The concept of an *invariant set* is that once the system is within the set, it can be kept within that set by the control or planning signals based on system dynamics. For collision avoidance scenarios, the desired set is space where the distance to the nearest object (and relative approach speed) is maintained sufficiently large. In the case of geo-fencing applications the desired invariant set is anywhere other than the geo-fenced region. For a system that may fall over, the desired state might be one where it remains upright.

Theory and practical uses are described in:

Ames, Aaron D., et al. "Control barrier function based quadratic programs for safety critical systems." IEEE Transactions on Automatic Control 62.8 (2016): 3861-3876.

Chen, Yuxiao, et al. "Backup control barrier functions: Formulation and comparative study." 2021 60th IEEE Conference on Decision and Control (CDC). IEEE, 2021.

Gurriet, Thomas. "Applied safety critical control." PhD diss., California Institute of Technology, 2020.

Singletary, Andrew, Shishir Kolathaya, and Aaron D. Ames. "Safety-critical kinematic control of robotic systems." IEEE Control Systems Letters 6 (2021): 139-144.

The basic concept is to use the current state of a dynamical system (robot arm, mobile device, aircraft, marine vessel, etc.) and a predetermined set of possible actions to drive a model of that system to predict when an undesirable condition will occur. Inputs including locations, geometries, speeds, and accelerations of obstacles are also needed when the CoPilot is designed for collision avoidance. The approach predicts what possible actions would lead to keeping the robot in a desirable configuration (e.g. a desirable input set), and then to modify the currently requested steering/speed/attitude commands to use the closest values in the desirable input set. The CoPilot then publishes modified inputs to slow or divert the device away from the collision path. The families of possible actions are built into the CoPilot by 3Laws based on the objective for the particular deployment.

Set-invariant theories are implemented through Control Barrier Functions (CBFs) which can be used to describe the desired state set (e.g. the "safe" set). It is typically not possible to come up with

an explicit expression to describe the desired invariant set, so some alternative approaches to enforce the same concepts have been developed. The CBFs also provide requirements on what conditions the desired inputs must satisfy to keep the system state inside the target space. Those requirement involve combining the derivatives of the CBFs with respect to the state variables and the equations of motion of the original system. The resulting expression is a multi-dimensional inequality which can be solved through Quadratic Programming. The equation of motion of the system is a function (typically nonlinear) of the current system state and of the inputs to the system. Since the possible acions would be used as control commands the system, one can evaluate if a particular choice satisfies the relationships that will result in keeping the state inside the target set/space.

3.6 CoPilot Operational Modes

Understanding of the discussion in this section is not necessary for use of the off-the-shelf configurations that are provided. These operational modes are pre-programmed into the software. If the platform or application is not one of the options discussed above the modes below are options that 3Laws will consider when building a new application/platform.

Based on the physical system being used and the desired operation conditions, the CoPilot has multiple methods to produce solutions determine the best failsafe strategy to use at any time. CoPilot currently supports the following methods, but 3Laws has already selected the most appropriate for the dynamical systems/applications that is has implemented.

Explicit: For simple physical systems it is possible to construct analytical functions. For example, if the goal is to keep an object within a box that spans x=[-1,1] and y=[-1,1], the barrier function (inequalities) can be x^2-1 >= 0 and y^2 - 1 >= 0. With an explicit barrier function and the equation of motion for the system, various failsafe strategies can be evaluated for compliance with the needs.

One can use a QP solver to find the command that best keeps the vehicle in the desired region.

A problem with the explicit approach is that if the system reaches the boundary of the safety set, then the desired input from the planner is ignored because the failsafe is the only strategy that is applied. For example, this might result in a condition where a request to back away from an obstacle is not allowed to happen.

Explicit smart switching has heuristic-based approaches to avoid the problem of getting stuck. The computation carries along several failsafe strategies. If one of the strategies can drive the system away from the boundary better than the others, that strategy is applied. Once the system is no longer at the boundary of the safe region, motion requests from the planner are applied instead of being overridden.

Implicit: Another approach is to create a family of available actions ahead of time. These actions are propagated to develop the set of actions that will keep the device in the desired space and which will not. Next, an optimization is made to find the commands in the desired space that are closest to the desired input commands. Note that if the current desired inputs are already in the desired set, then there will be no changes to those inputs. An interesting feature of this approach is that the approach starts pushing away from the raw desired inputs when the desired inputs

> begin to violate the desired objectives.

3Laws won't know how far the robot is from the edge of the control invariant set, but the code can measure the distance to the edge of the original "safety" set.

When integrating over the space, the approach also integrates the sensitivity. The sensitivity gives information used to compute the optimally close (to the original) inputs. The sensitivity at each point is the effect of changing the action at the beginning of the integration. The edge of the control invariant safety set is described by the collection of multiplying the gradients of the full safety sets times the gradient of the equation of motion times the sensitivity over the horizon of integration. This results in a scalar constraint for each step that must be greater than zero. These work as constraints on a quadratic problem that is searching for the best failsafe strategy to apply.

Implicit with switching: To make the system less prone to getting stuck when using the implicit approach, a larger family of possible actions can be used to calculate the various forward integrations.

This ends up being computationally costly, so algorithms have been created to switch between possible modifications to produce a good failsafe for the current step.

Additional parameters will be added based on the equations of motion for the individual system.